

Capacity Planning Tool — Design Document

1. Purpose

This prototype replaces spreadsheet-based strategic capacity planning with a web-based tool that provides consistent, comparable, and up-to-date visibility into team capacity, initiative delivery, and cross-team impact of new demands.

The tool operates strictly at **initiative** and **epic** level.

2. Stakeholders & Needs

Higher Management

- Portfolio-level visibility across all teams
- Identify overcommitted teams and risks
- Understand impact of adding new initiatives

Engineering Managers / Product Managers

- Plan initiatives across multiple teams
- Compare scenarios before committing
- Make trade-off decisions based on capacity

Team Leads / Product Owners

- Model realistic team capacity
- Account for availability, overhead, and support load
- Understand how many epics fit into a planning period

3. Architecture Overview

Frontend

- React + TypeScript + Vite
- Pages: Dashboard, Teams, Initiatives, Epics
- REST API integration
- Lightweight global CSS (App.css)

Backend

- Spring Boot (Java 17)
- REST API
- Service layer for capacity calculation
- Repository layer for persistence

Database

- PostgreSQL
- Stores teams, persons, initiatives, epics, planning periods

Runtime

- Docker Compose (frontend + backend + db)

4. Domain Model

Team

- id, name
- overheadPercentage
- supportLoadPercentage
- workingDaysPerPeriod

Person

- id, name
- teamId
- availabilityPercentage

Initiative

- id, name
- estimatedEffort (person-days)
- targetDate

Epic

- id, name
- initiativeId
- teamId
- estimatedEffort
- dueDate
- status

PlanningPeriod

- id, name (e.g., 2025-Q1)
- startDate, endDate

Computed: TeamCapacity

- availableDays
- allocatedDays
- utilizationPercentage
- status (OK / AT_RISK / OVERCOMMITTED)

5. Capacity Calculation Logic

Base capacity

1. $\text{teamSize} \times \text{workingDaysPerPeriod}$

Apply availability

2. Sum of $(\text{workingDaysPerPeriod} \times \text{availabilityPercentage})$ per person

Apply overhead & support

3. $\text{effectiveDays} \times (1 - \text{overhead}) \times (1 - \text{supportLoad})$

Allocated days

4. Sum of epic estimates for that team in that period

Utilization

5. $\text{allocatedDays} / \text{projectCapacity} \times 100$
6. **Status thresholds**
 - $< 90\% \rightarrow \text{OK}$
 - $90-110\% \rightarrow \text{AT_RISK}$
 - $> 110\% \rightarrow \text{OVERCOMMITTED}$

6. API Overview

- `GET /api/dashboard` → summary counts
- `GET /api/teams`
- `GET /api/teams/{id}/capacity?periodId=`
- `GET /api/initiatives`
- `GET /api/initiatives/{id}/epics`
- `GET /api/epics?teamId=&periodId=`

7. UI Overview

Dashboard

- High-level metrics: Teams, People, Initiatives, Epics
- Team capacity table: available, allocated, utilization, status

Teams Page

- Team details + capacity view

Initiatives Page

- List of initiatives + linked epics

Epics Page

- List of epics with team, estimate, due date

8. Assumptions (Ambiguous Requirements Clarified)

Effort units

1. All estimates use **person-days**.

Planning periods

2. Predefined (e.g., quarters). No automatic calendar calculation.

Holidays & vacations

3. Not modeled individually; captured via availability and overhead.

Initiative → Team mapping

4. Initiatives span teams **via epics**, not directly.

Scheduling

5. Prototype does not auto-schedule epics; user assigns periods manually.

Simulations

6. Stateless: parameters changed in UI → backend recalculates capacity without saving scenarios.

Security

7. No authentication/authorization in prototype.

Performance

8. Dataset small; no caching or pagination required.

Styling

9. Minimal global CSS; not a design-system implementation.

Data completeness

10. Prototype uses seed data; no complex validation or workflows.

9. Future Enhancements

- Holiday calendars per region
- Role-based overhead
- Scenario saving & comparison
- Authentication & RBAC
- Advanced visualization (burn-up, forecast curves)